Operating Systems Lab — Assignment 7

Q1 — User threads and an unsynchronized counter

The user API is `clone(void (*fcn)(void *), void *arg, void *stack)` with `join()` for waiting and cleanup. The wrapper places the function and argument on the supplied stack; the kernel extracts the three arguments in that order and starts a thread sharing its parent's address space. Three threads each attempt 100,000 increments without synchronization. The final three-hart run printed:

```
Q1 unsynchronized counter=300000 expected_without_race=300000
```

This is the observed result, not a guarantee: the counter's read-modify-write can lose updates. Each thread has a separate trapframe mapping despite the shared page table. On `join()`, cleanup unmaps that thread's trapframe and reclaims its page; thread exit also closes its duplicated file and cwd references.

Q2 — Semaphores and producer/consumer

Three threads each perform 100,000 increments protected by `sem_wait()` and `sem_signal()`. The producer and consumer use a five-slot circular buffer, an empty-slots semaphore, a full-slots semaphore, and a buffer mutex. The program checks both overflow and underflow. The latest three-hart xv6 run printed:

```
Q2 PASS counter=300000 buffer=0 overflow=0 underflow=0
```

The final bounded three-hart xv6 run printed the PASS line above and emitted produced/consumed item and buffer-slot logs. A separate logging semaphore serializes each full line because xv6 `printf()` writes characters separately. Two genuine terminal captures show a portion of the item logs and the final PASS result.

Q1 and Q2 both attempted 300,000 increments. Q1 happened to retain all updates in its measured run but remains racy; Q2 asserts the exact count under semaphore protection. The producer/consumer buffer was empty at completion, with zero overflow and underflow.

The semaphore interface matches the handout: `sem_init(int sem_id, int value)`, `sem_wait(int sem_id)`, and `sem_signal(int sem_id)`. The kernel holds 64 globally indexed semaphore entries, each with an initialized flag, count, and lock. Wait sleeps when the count is zero; signal increments the count and wakes a waiter. Q2 uses IDs 0–3 for the counter mutex, empty slots, full slots, and buffer mutex, plus ID 4 to serialize producer/consumer log lines.

Q3 — Banker's safety, matrix detection, and process circular wait

Banker's Algorithm checks declared maximum claims and allocation to find a safe completion sequence. An unsafe state is not, by itself, proof of a current deadlock. Matrix deadlock detection instead checks current allocation, outstanding requests, and available resources. `deadlock.c` prints the intermediate work/remaining-resource vectors. The bounded actual-process demonstration separately forks two children: each holds a token from one pipe and waits for a token from the other; the parent releases both to wait, pauses briefly, then kills and reaps them.

The final one-hart run printed:

```
BANKER available: 3 3 2
BANKER after P1 finishes, work: 5 3 2
BANKER after P3 finishes, work: 7 4 3
BANKER after P4 finishes, work: 7 4 5
BANKER after P0 finishes, work: 7 5 5
BANKER after P2 finishes, work: 10 5 7
CASE 1 PASS: Banker's safe sequence: P1 P3 P4 P0 P2
BANKER available: 0 0 0
BANKER unsafe; remaining work: 0 0 0
CASE 2 PASS: Banker's unsafe state has no safe sequence
CASE 3 initial state
Available: 1 0 0
P0 Allocation: 1 0 0
P0 Need: 1 0 0
P1 Allocation: 1 0 0
P1 Need: 2 0 0
BANKER after P0 finishes, work: 2 0 0
BANKER after P1 finishes, work: 3 0 0
CASE 3 initial state PASS: safe
CASE 3 hypothetical request P1: 1 0 0
CASE 3 request validity: 1 <= Need[1]=2 and Available=1
State after hypothetical grant
Available: 0 0 0
P0 Allocation: 1 0 0
P0 Need: 1 0 0
P1 Allocation: 2 0 0
P1 Need: 1 0 0
BANKER unsafe; remaining work: 0 0 0
BANKER REQUEST for P1 is unsafe; do not grant
CASE 3 PASS: valid hypothetical request makes safe state unsafe
DETECT available: 0 0
DETECT after P0 finishes, work: 1 0
DETECT after P1 finishes, work: 1 1
DETECT unfinished: none
DETECT remaining work: 1 1
CASE 4 PASS: deadlock detector found no deadlock
DETECT available: 0 0
DETECT unfinished: P0 P1
DETECT remaining work: 0 0
CASE 5 PASS: deadlock detector identified P0 and P1
ACTUAL-PROCESS CASE: P4 holds A/waits for B; P5 holds B/waits for A
ACTUAL-PROCESS CASE PASS: circular wait was bounded and terminated
```

Cases 1–2 are Banker's safe/unsafe examples. Case 3 starts safe, then considers a valid request from P1 for one unit: it is within P1's Need (2) and Available (1). After the temporary grant, Available is zero and both processes need one unit, so no process can finish. The grant is rejected as unsafe; the original state is retained. Cases 4–5 are matrix no-deadlock/deadlock detection. The last two lines report the independent real-process circular-wait test. PIDs vary between runs.

Q4 — Priority scheduling, aging, and measured completion

Valid priorities are 1–20 (1 is highest); new processes default to 10. The preemptive scheduler selects the runnable process with the lowest numeric priority and rotates its scan origin for round-robin tie-breaking. On each global timer tick, a RUNNABLE process's wait counter increments; after it is greater than 30, priority improves by one and the counter resets. The `struct proc` scheduling fields hold the current priority, minimum effective priority observed, and wait ticks. `getpriority(pid)` returns that observed minimum to the process or its parent.

Q4 starts exactly four child processes at distinct priorities 1, 5, 10, and 20. All run the same CPU-bound workload (900,000,000 loop iterations). Children write their PID, initial priority, elapsed ticks, and observed minimum effective priority to a pipe; the parent records and prints their actual completion order. The parent also polls the priority-20 child to report observed aging. Completion ticks are measured from the start of the experiment, including process creation and queueing. The latest three-hart run printed:

```
Q4 started pid=4 initial_priority=1
Q4 started pid=5 initial_priority=5
Q4 started pid=6 initial_priority=10
Q4 started pid=7 initial_priority=20
Q4 aging observed pid=7 initial_priority=20 effective_priority=19
Q4 aging observed pid=7 initial_priority=20 effective_priority=18
Q4 aging observed pid=7 initial_priority=20 effective_priority=17
Q4 completion order=1 pid=4 initial_priority=1 elapsed_ticks=116 effective_priority=1
Q4 completion order=2 pid=5 initial_priority=5 elapsed_ticks=118 effective_priority=5
Q4 completion order=3 pid=6 initial_priority=10 elapsed_ticks=118 effective_priority=10
Q4 completion order=4 pid=7 initial_priority=20 elapsed_ticks=186 effective_priority=17
Q4 PASS: priority 20 reached effective 17
```

Completion order	PID	Initial priority	Elapsed ticks	Minimum effective priority
1	4	1	116	1
2	5	5	118	5
3	6	10	118	10
4	7	20	186	17

This run shows observable aging and completion of all four priority classes. The values are uptime-tick measurements from one run, not timing guarantees. There are exactly four workload children; the start/completion PID records associate each measured value with its process.

Design decisions, problems, and solutions

- Threads share a page table, so using one trapframe virtual address caused

multi-hart return-to-user collisions. A unique per-process-slot trapframe mapping fixed the observed startup fault.

- Each thread duplicates file and cwd references at clone. Thread exit now

releases those references; `join()` unmaps and reclaims the thread trapframe without freeing the shared address space.

- Q2 uses the specified integer-ID kernel semaphore API for every shared

counter update; it does not substitute an atomic spin lock. Separate empty/full/mutex IDs enforce producer-consumer capacity and availability.

- Q3 prints Available, Allocation, and Need for the hypothetical Banker

request before and after granting it to a copy of the state. The safe initial state and unsafe post-request state are therefore compared directly.

- The first Q4 test used uneven workloads and more than four children, making

completion comparisons misleading. It now uses four equal CPU-bound workloads and reports the observed PID, priority, order, elapsed ticks, and effective priority for each.

Build and verification

The existing xv6 build and filesystem contain the current sources and user binaries. Q2 and Q3 were run in bounded interactive QEMU sessions for these captures; both printed their passing results. Q4 printed its passing result in a bounded three-hart run. `make user/_semtest` also compiled the separate semaphore test against the integer-ID API.

Program	Configuration	Observed outcome
`threadtest`	3 harts	300,000 observed; all three workers joined
`synctest`	3 harts	PASS: 300,000; buffer 0; overflow 0; underflow 0
`deadlock`	1 hart	All Banker, matrix-detection, and bounded-process cases passed
`priotest`	3 harts	Four equal-work children completed; priority 20 aged to 17

`Makefile` registers all four programs. The packaged Q2 source matches the working source. Q2's PASS result and serialized per-item producer/consumer logs were observed in the latest run.

Screenshot status and reproduction

Q1 remains the existing thread-test capture. Q2 uses two genuine captures: `synctest_logs.png` shows produced/consumed item and slot logs, while `synctest.png` shows later logs and the successful 300,000 counter result with zero buffer errors. Q3 uses `deadlock_1.png` and `deadlock_2.png` because the complete output does not fit legibly in one screen. Q4 is a current capture showing four children, observed aging from 20 to 17, all completion records, and PASS.

From the source tree, after closing any existing QEMU instance:

```
cd /home/ajay/os-lab/xv6-riscv
make kernel/kernel
make fs.img
make CPUS=3 qemu
```

At the xv6 prompt run `threadtest` and `synctest`. Restart with `make CPUS=1 qemu` and run `deadlock`; restart with `make CPUS=3 qemu` and run `priotest`.

The Q2 images show actual output from the final source, including serialized producer/consumer logs and the PASS line. The two Q3 images together show the complete run. The Q4 image records the actual values printed by the latest run.